

Submitting your project

For this project, you must hand in:

- battery.py

You may resubmit any time before the deadline. Your last submission will be graded for correctness and will be the version that the TA sees. Submissions must be done using MarkUs.

Main block

All testing code must be in the `if __name__ == '__main__':` block. All global variables must be initialized outside of this block (i.e., in the `initialize()` function).

Clarifications

Any clarifications will be posted to Piazza. You are responsible for monitoring this discussion board, particularly the announcements.

How you will be marked

Your work will be marked for correctness using MarkUs and for your understanding during the project demo.

Autotesting for Correctness

Your functions need to work as described below. We will run your functions using a Python 3 interpreter. Please confirm you are using Python 3 as well. To check what version of Python you are running, you can run the following in your Python shell:

```
import sys
sys.version
```

To make your code for correctness, you will need to upload your code to the MarkUs platform. Since the code is graded automatically, it is critical that the code does not contain syntax errors, as this will cause your mark to be 0. If your file contains syntax errors, you will see that on MarkUs right away. In addition, your functions must be implemented precisely as described. Their signature must exactly match the handout and their behaviour must be exactly as specified. This means that the functions should not print anything unless the project specifications require that and that they should return exactly what the project handout describes.

Some of the tests used to evaluate your project can be run by you. However, the tests we provide you with are not exhaustive. We will test your code with more test cases for evaluation. As a result, you should test your code yourself to be confident it works.

We encourage you to upload your code to MarkUs early to ensure you know how to do so, potentially even before completing all of the functions. Note, MarkUs is currently set to allow you to run tests on your code 12 times per hour.

Project Demo

You will be assigned a 15-minute assessment time on Wednesday, October 14, during your assigned lab period. You will demo your project to a TA, who will ask you questions about your code and may ask you to write/modify some code on the spot. Although you are working in pairs, you will receive an individual mark based on your individual performance during the project demo. You will be expected to arrive at least 5 minutes before your scheduled project demo time to set up your project code. Once you have completed your demo, you are welcome to leave.

Documentation

Note that you will not be assessed on documentation or code style for this project. However, having clear comments could help you navigate your own code and answer questions more easily. The following are some instances of when you should add comments to your code:

- For every function, as a docstring, to describe the parameters of the function and what the function does. See below for more details on docstrings.
- Before every global variable declaration, to describe what kind of information the variable stores and what properties (if any) that information is supposed to have throughout the execution of the code.
- Before all complicated sections of code, to help the reader understand what that code section is trying to do.
- In general, comments should not simply restate what the code does (this does not add any useful information to the code). Comments should add information that is implicit in the code, e.g., about what purpose a computation serves, or why a certain section of code is written the way it is.

Guidelines for writing docstrings

- Describe precisely *what* the function does.
- Do not reveal *how* the function does it.
- Make the purpose of every parameter clear.
- Refer to every parameter by name.
- Be clear about whether the function returns a value, and if so, what.

- Explain any conditions that the function assumes are true. Examples: “n is an int”, “n != 0”, “the height and width of p are both even”.
- Be concise.
- Ensure that the text you write is grammatically correct.
- Write the docstring as a command (e.g., “Return the first...”) rather than a statement (e.g., “Returns the first...”).

Style

Good style practices should be adhered to when writing your code. This includes the following:

- Use Python style conventions for your function and variable names. In particular, please use “pothole case”: lowercase letters with words separated by underscores (`_`), to improve readability.
- Choose good names for your functions and variables. For example, `num_coffee_cups` is more helpful and readable than `ncc`.
- To make sure your program will be formatted correctly is never to mix spaces and tabs when indenting—use only tabs, or only spaces.
- Put a blank space before and after every operator. For example:

```
b = 3 > x and 4 - 5 < 32 # good style: easy to read
```

```
b= 3>x and 4-5<32 # bad style: hard to read
```
- Write a docstring comment for each function. (See above for guidelines on the content of your docstrings.) Put a blank line after every docstring comment.
- Each line must be less than 80 characters long, including tabs and spaces. You should break up long lines using `\`.
- Your code should be readable and readily understandable.

Many modern technologies rely on rechargeable batteries, including electric vehicles, e-bikes, laptops, smartphones, and clean energy systems. As systems transition to renewable energy sources such as wind and solar power, the need to store energy becomes more crucial. Further, the electric vehicle industry relies on rechargeable batteries. Research continues to investigate ways to improve this technology, including improving their performance and extending their lifetime.

In this project, you will implement a simulator for a rechargeable battery. Note, many aspects of this assignment are a substantial simplification compared to how batteries actually work but are inspired by real properties.

The simulation proceeds as a series of operations, which are calls to functions you will define. For example, a simulation might proceed as follows:

```
if __name__ == '__main__':
    initialize()
    simulate_activity('charge', 30)
    print(get_cur_temp())
    print(get_cur_charge())
    print(duration_fast_charge_possible())
    simulate_activity('idle', 10)
    simulate_activity('use', 25)
    print(get_cur_temp())
    print(get_cur_charge())
    print(charge_time_needed(40))
```

The lines of code correspond to different activities (i.e., charging the battery, leaving the battery idle, using the battery to power some technology), querying for the temperature and charge status, and querying for the charge time needed for future use.

The following rules apply to how the battery charge, temperature and health are updated:

- Initial battery charge is set by the `initialize()` function. Battery begins in good health, at 50% charge and at 20°C temperature.
- The battery is always charging, being used, or sitting idle.
- Charging can be fast or slow. Fast charging increases the charge by 3% per minute, slow charging by 1% per minute.
- Fast charging occurs when the temperature is between 0-40°C, battery charge is below 80%, and battery health is good.
- Slow charging increases the temperature by 0.25°C per minute. Fast charging increases the temperature by 0.5°C per minute.
- If fast charging is possible, it will occur. Charging can switch from fast to slow charging in a single session.

- Usage discharges the battery by 2% per minute. Temperature increases by 1°C per minute during usage.
- Sitting idle discharges the battery by 0.5% per minute and decreases the temperature by 1°C per minute. The temperature can never go below 0°C.
- If the battery charge reaches 0% during usage or idling, its temperature will decrease by 1°C per minute that it is dead (not going below 0°C), while the charge does not decrease past 0%.
- Charging at or past 90% reduces the battery health. If this happens 3 times within 6 hours, the battery is in a bad health state.
- Batteries in bad health state cannot charge past 80%. If the battery initially switches to a bad health state, it will not charge further until discharged below 80%.
- If the battery charge reaches 100% during charging, its temperature will continue to increase by 0.25°C per minute, while the charge does not increase past 100%. The same logic applies to if the battery is in a bad health state and reaches 80% during charging, or if the battery switches to a bad health state and is at or above 90%: the temperature will increase by 0.25°C per minute, while the battery charge will not increase further.

Functions to Implement

We have provided you with a starter version of `battery.py`, with some parts already filled in. Please read carefully and make sure everything makes sense before you make changes! You must not change any of the function signatures.

Part (a) `get_cur_temp()`

This function returns the current temperature of the battery, as a float.

Part (b) `get_cur_charge()`

This function returns the current charge level of the battery, as a float.

Part (c) `get_cur_battery_health()`

This function returns the health of the battery, as a boolean. It returns True if the battery is in good health and False if the battery is in bad health.

Part (d) `duration_fast_charge_possible()`

This function returns the duration for which fast charge is possible based on the current battery health, temperature, and charge as a starting point.

Part (e) `simulate_activity(activity, duration)`

This function simulates the battery performing the activity for duration minutes. Assume duration is a positive int. If the activity is not one of “charge”, “use”, or “idle”, running the function should have no effect.

Part (f) `charge_time_needed(minutes)`

This function returns the duration needed for charging to enable use for a specific activity duration of `minutes` afterwards. Based on a future journey, how long does the battery need to charge now to have sufficient battery for `minutes` use duration? If the battery already has sufficient charge, return 0. If it is impossible for the battery in its current state to be charged to a point where usage for `minutes` duration can be performed, return `None`.

Part (g) `initialize()`

This function initializes all the global variables in the program. The following code should run two independent simulations, with both SIMULATION 1 and SIMULATION 2 starting from the beginning.

```
initialize()  
# SIMULATION 1 CODE  
# ...
```

```
initialize()  
# SIMULATION 2 CODE  
# ...
```

FAQ

What is meant by % in the battery charge? Is the charging rate based on a percentage of the current charge level?

The % sign is treated as a unit here. You can ignore it in your simulations and add up the values. For example, if a battery is at 50% charge and increases by 10%, the battery will be at 60% charge (not 55%).

Can the battery charge ever be below 0% or above 100%?

No. If there is a situation that would lead to this occurring, the battery charge will not continue to increase past 100% or decrease below 0%. The temperature and time will still be incremented according to the rules set out above.

If charging exceeds 90% during charging, at what time is it considered to be overcharged?

The minute that the battery charge reaches 90% is the minute that the overcharge has occurred. Let's say we charge a good-health battery for 11 minutes with a starting charge of 80%. Slow charging occurs for 11 minutes such that the ending charge is 91%, so an overcharge has occurred at minute 10 of charging.

What happens when a battery is placed in a bad health state during a charging session and is already charged past 80%?

The battery charge will be stuck at its current battery level for the remainder of the charging session. No charging activity can increase the battery charge further until it is discharged below 80%, after which charging cannot increase the battery charge beyond 80%.

If the battery charge is already at or above 90% and an additional charging activity is initiated, does it count as an overcharge?

Yes.

Is it possible to go from slow to fast charging within the same charging session?

No.

If the battery is in a bad health state, is it possible for it to revert back to a good health state?

No, not for the remainder of the simulation. The only way to “reset” the battery would be to start a new simulation by calling `initialize()`.